A Hybrid Grey Wolf Optimizer–Simulated Annealing Algorithm for Multi-Objective Task Scheduling in Cloud Computing

Aditya Shukla
Dept. of Electrical Engineering
IIT (BHU) Varanasi
Roll No: 24085003
aditya.shukla.eee24@iitbhu.ac.in

Divya Khandelwal
Dept. of Electrical Engineering
IIT (BHU) Varanasi
Roll No: 24085023
divya.khandelwal.eee24@iitbhu.ac.in

Nirupam Mahanta
Dept. of Electrical Engineering
IIT (BHU) Varanasi
Roll No: 24085059
nirupam.mahanta.eee24@iitbhu.ac.in

Prof. Prasenjit Chanak
Associate Professor, Dept. of Computer Science and Engineering
Indian Institute of Technology (BHU) Varanasi
prasenjit.cse@iitbhu.ac.in

***Abstract*—Cloud computing task scheduling is a well-known NP-hard optimization problem. It requires efficient allocation of tasks to virtual machines (VMs). The allocation must minimize makespan, cost, and workload imbalance simultaneously. This paper proposes Hybrid GWO-SA, a novel metaheuristic algorithm. It combines Grey Wolf Optimizer (GWO) for global exploration with Simulated Annealing (SA) for local refinement. Three key innovations are introduced: (i) smart population seeding using FCFS and Min-Min heuristics, (ii) bottleneck-guided SA refinement of the top-three wolf leaders, and (iii) periodic diversity injection through worst-population re-initialization. The proposed algorithm is benchmarked against nine algorithms. These include three classical heuristics, three pure metaheuristics, and three state-of-the-art hybrid algorithms from 2025. Four workload scenarios ranging from 100 to 1,000 tasks are tested. Results show that Hybrid GWO-SA achieves the lowest makespan in all scenarios. Improvements reach 55.50% over WOA-GS, 26.62% over PSO-GWO, and 34.45% over DECSO. The Degree of Imbalance is reduced to 0.013–0.044. This represents up to 98.61% improvement in load fairness. Statistical validation over 30 Monte Carlo runs confirms significance at $p < 0.001$.**

***Index Terms*—Cloud computing, task scheduling, Grey Wolf Optimizer, Simulated Annealing, hybrid metaheuristic, multi-objective optimization, load balancing, makespan minimization, NP-hard, CloudSim Plus**

I. Introduction

Cloud computing is the dominant paradigm for scalable resource provisioning [1]. A critical challenge in cloud data centers is task scheduling. Task scheduling assigns n independent tasks to m heterogeneous VMs. Multiple conflicting objectives must be optimized simultaneously. These objectives include execution time, monetary cost, and workload balance. The problem is NP-hard [2]. Exact solutions are computationally intractable for large instances. The search space grows as m^n, which exceeds 10^{100} for moderate problem sizes.

Classical heuristics provide deterministic solutions in polynomial time. Round Robin, FCFS, and Min-Min are commonly used approaches [3]. However, these heuristics produce suboptimal results. Their greedy or cyclic nature limits solution quality.

Metaheuristic algorithms offer stochastic exploration of the search space. The Grey Wolf Optimizer (GWO) [4] emulates wolf pack hunting behavior. It excels at global exploration through hierarchical leadership. Simulated Annealing (SA) [5] provides robust local search. SA escapes local optima through temperature-controlled probabilistic acceptance. Particle Swarm Optimization (PSO) [6] uses swarm intelligence. Each method has individual strengths but also specific weaknesses.

Recent literature shows a surge in hybrid metaheuristic approaches. Acharya *et al.* [**?**] proposed WOA with Golden Section search. Kumari *et al.* [**?**] introduced a PSO-GWO hybrid. Gündüz and Gürkaş-Aydın [**?**] presented DECSO. These approaches show improvements over standalone methods. However, they exhibit limitations in balancing exploration and exploitation. Performance degrades at larger problem scales.

This paper proposes **Hybrid GWO-SA**. It addresses the limitations through three architectural innovations:

1) **Smart Population Seeding:** Two wolves are initialized using FCFS and Min-Min heuristics. This provides high-quality seed solutions from iteration zero.
2) **Bottleneck-Guided SA Leader Refinement:** The top three wolves undergo SA refinement. Task migration targets the most loaded VM directly.
3) **Periodic Diversity Injection:** The worst 20% of the population is re-initialized every 25 iterations. This prevents premature convergence.

The proposed algorithm is evaluated across four cloud scenarios. These range from 100 tasks with 10 VMs to 1,000 tasks with 50 VMs. Nine baseline algorithms serve as benchmarks. Results are validated using 30-run Monte Carlo statistical testing.

II. Literature Review

A. Classical Scheduling Heuristics

Round Robin assigns tasks cyclically without considering heterogeneity. This leads to severe load imbalance [3]. FCFS allocates tasks to the least-loaded VM at arrival time. It provides moderate balance but ignores global optimization. Min-Min [7] assigns the shortest task to the fastest VM. It achieves better makespan but starves long tasks. These heuristics run in polynomial time. However, they cannot optimize multiple objectives jointly.

B. Metaheuristic Approaches

GWO [4] models the α-β-δ-ω wolf hierarchy. A decreasing exploration coefficient a transitions the search phase. However, GWO suffers from premature convergence in discrete spaces. SA [5] provides local search with Boltzmann acceptance. The acceptance probability is $P = \exp(-\Delta F/T)$. SA lacks population-based global exploration capability. PSO [6] uses cooperative swarm search. It stagnates in multimodal discrete landscapes. Genetic Algorithms (GA) [8] use selection and crossover. GA has high computational overhead for online scheduling.

C. State-of-the-Art Hybrid Approaches (2025)

Acharya *et al.* [**?**] combined WOA with Golden Section. The method targets fog-cloud scheduling environments. Improvements of 15–20% over standalone WOA are reported. Kumari *et al.* [**?**] hybridized PSO with GWO hierarchy. The method improves convergence on medium-scale instances. Gündüz and Gürkaş-Aydın [**?**] proposed DECSO. It integrates differential evolution with chicken swarm optimization. Competitive performance on heterogeneous workloads is achieved.

D. Research Gaps and Contributions

Existing hybrids lack three critical features. First, problem-specific initialization using scheduling domain knowledge is absent. Second, targeted local search guided by workload bottleneck analysis is missing. Third, systematic diversity management mechanisms are not implemented. The proposed Hybrid GWO-SA addresses all three gaps.

Table I provides a comparative summary of the reviewed approaches.

III. System Model and Problem Formulation

A. Cloud Computing Architecture

Fig. 1 illustrates the cloud computing system model. The architecture consists of three layers. The user layer generates computational task requests. The broker layer runs the scheduling algorithm. The datacenter layer hosts heterogeneous VMs on physical machines.

The task scheduler receives n independent tasks. It produces an optimal task-to-VM mapping. The mapping is deployed to the cloud datacenter for execution.

B. Task and VM Model

Table II lists all mathematical symbols used in this paper.

A set of n independent tasks $\{T_1, T_2, \ldots, T_n\}$ is considered. Each task T_i has a computational workload w_i measured in Million Instructions (MI). Tasks are independent with no inter-task data dependencies.

A set of m heterogeneous VMs $\{V_1, V_2, \ldots, V_m\}$ is available. Each VM V_j has processing speed p_j in MIPS and cost rate c_j in \$/second. The execution time of task T_i on VM V_j is:

$$e_{ij} = \frac{w_i}{p_j} \tag{1}$$

The scheduling problem finds a mapping $\sigma : \{1, \ldots, n\} \to \{1, \ldots, m\}$. Each task T_i is assigned to exactly one VM $V_{\sigma(i)}$.

C. Objective Functions

Three conflicting objectives are optimized simultaneously.

1) Makespan: The completion time of VM V_j is the sum of execution times of all assigned tasks:

$$\mathrm{CT}_j = \sum_{i:\sigma(i)=j} \frac{w_i}{p_j} \tag{2}$$

The makespan is the maximum completion time across all VMs:

$$f_1 = \max_{j=1}^{m} \mathrm{CT}_j \tag{3}$$

A lower f_1 indicates faster overall task completion.

2) Execution Cost: The total monetary cost is computed as:

$$f_2 = \sum_{j=1}^{m} c_j \cdot \mathrm{CT}_j \tag{4}$$

A non-linear tiered pricing model is adopted. Higher-MIPS VMs cost disproportionately more per unit of compute. This reflects real commercial cloud economics.

3) Degree of Imbalance: The workload fairness metric is defined as:

$$f_3 = \frac{\max(\mathrm{CT}_j) - \min(\mathrm{CT}_j)}{\mathrm{avg}(\mathrm{CT}_j)} \tag{5}$$

A value of $f_3 = 0$ indicates perfect load balance. Higher values indicate uneven workload distribution.

D. Multi-Objective Fitness Function

The three objectives are normalized using reference values. Normalization brings all objectives to a comparable scale:

$$\hat{f}_k = \frac{f_k}{f_k^{\mathrm{ref}}}, \quad k \in \{1, 2, 3\} \tag{6}$$

The reference values are derived from the problem instance. The reference makespan f_1^{ref} is the theoretical lower bound:

$$f_1^{\mathrm{ref}} = \frac{\sum_{i=1}^{n} w_i}{\max_j(p_j) \cdot m} \tag{7}$$

The reference cost f_2^{ref} uses the average cost rate:

$$f_2^{\mathrm{ref}} = \bar{c} \cdot f_1^{\mathrm{ref}} \cdot m, \quad \bar{c} = \frac{1}{m} \sum_{j=1}^{m} c_j \tag{8}$$

TABLE I: Comparative Summary of Related Scheduling Approaches

Reference	Year	Algorithm	Objectives	Limitations
Mirjalili *et al.* [4]	2014	Grey Wolf Optimizer (GWO)	Single-objective	Premature convergence in discrete spaces
Kirkpatrick *et al.* [5]	1983	Simulated Annealing (SA)	Single-objective	Single-solution method, slow exploration
Kennedy *et al.* [6]	1995	Particle Swarm Optimization	Single-objective	Velocity explosion, premature convergence
Acharya *et al.* [**?**]	2025	Hybrid WOA-GS	Makespan, cost	Golden Search inefficient in discrete space
Kumari *et al.* [**?**]	2025	Hybrid PSO-GWO	Makespan, balance	No explicit local trap escape mechanism
Gündüz *et al.* [**?**]	2025	Hybrid DECSO	Makespan, cost, DoI	Complex parameter tuning, poor scalability
Proposed	**2025**	**Hybrid GWO-SA**	**Makespan, cost, DoI**	**Higher runtime (acceptable trade-off)**

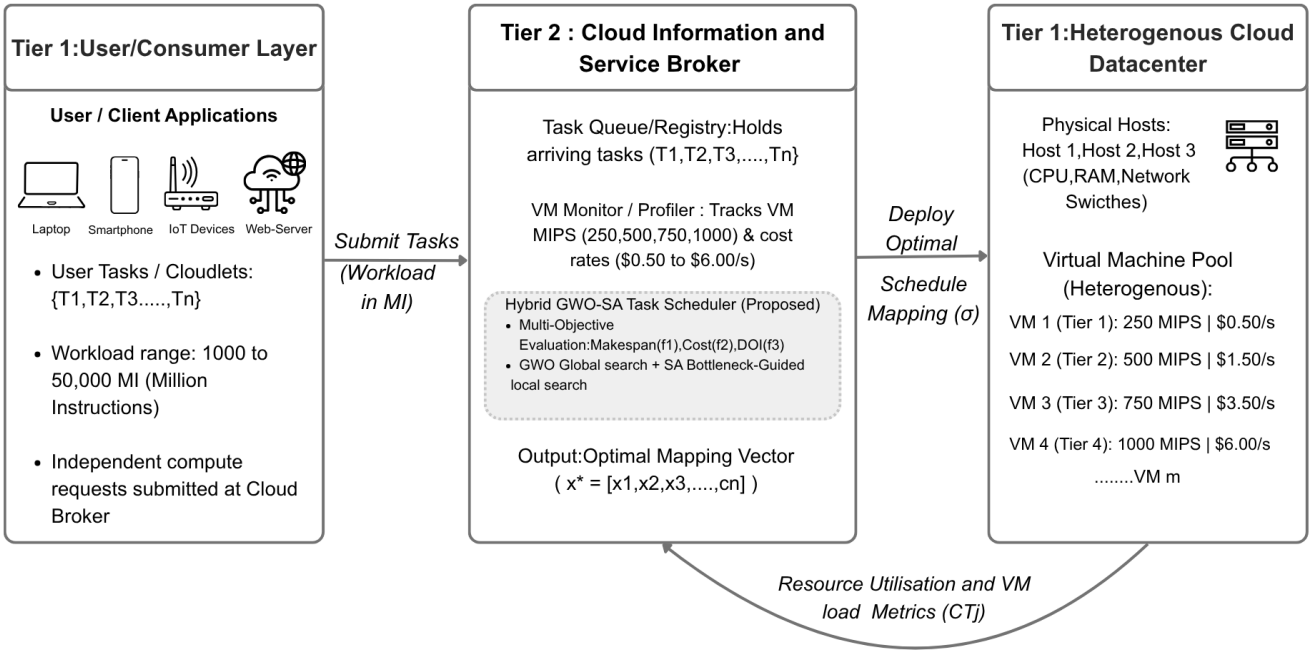


Fig. 1: Cloud computing network architecture. Users submit tasks to the cloud broker. The broker applies the Hybrid GWO-SA scheduler to assign tasks to heterogeneous VMs across physical hosts in the datacenter.

The reference DoI is set to $f_3^{\text{ref}} = 2.0$ based on empirical range.

The composite fitness function combines the normalized objectives:

$$F = \lambda_1 \cdot \hat{f}_1 + \lambda_2 \cdot \hat{f}_2 + \lambda_3 \cdot \hat{f}_3 \tag{9}$$

where $\lambda_1 = 0.5$, $\lambda_2 = 0.3$, and $\lambda_3 = 0.2$. Lower fitness F indicates a better schedule.

IV. Proposed Hybrid GWO-SA Methodology

A. Algorithm Overview

The Hybrid GWO-SA algorithm operates in a two-phase iterative loop. Fig. 2 presents the complete algorithmic flow.

In each generation, two phases execute sequentially. The **GWO phase** updates the wolf population using discrete leader-following. The **SA phase** refines the top-three leaders using bottleneck-guided search. This synergy combines GWO's hierarchical global search with SA's targeted exploitation.

B. Solution Representation

Each wolf (candidate solution) is encoded as an integer vector:

$$\mathbf{x} = [x_1, x_2, \ldots, x_n], \quad x_i \in \{0, 1, \ldots, m-1\} \tag{10}$$

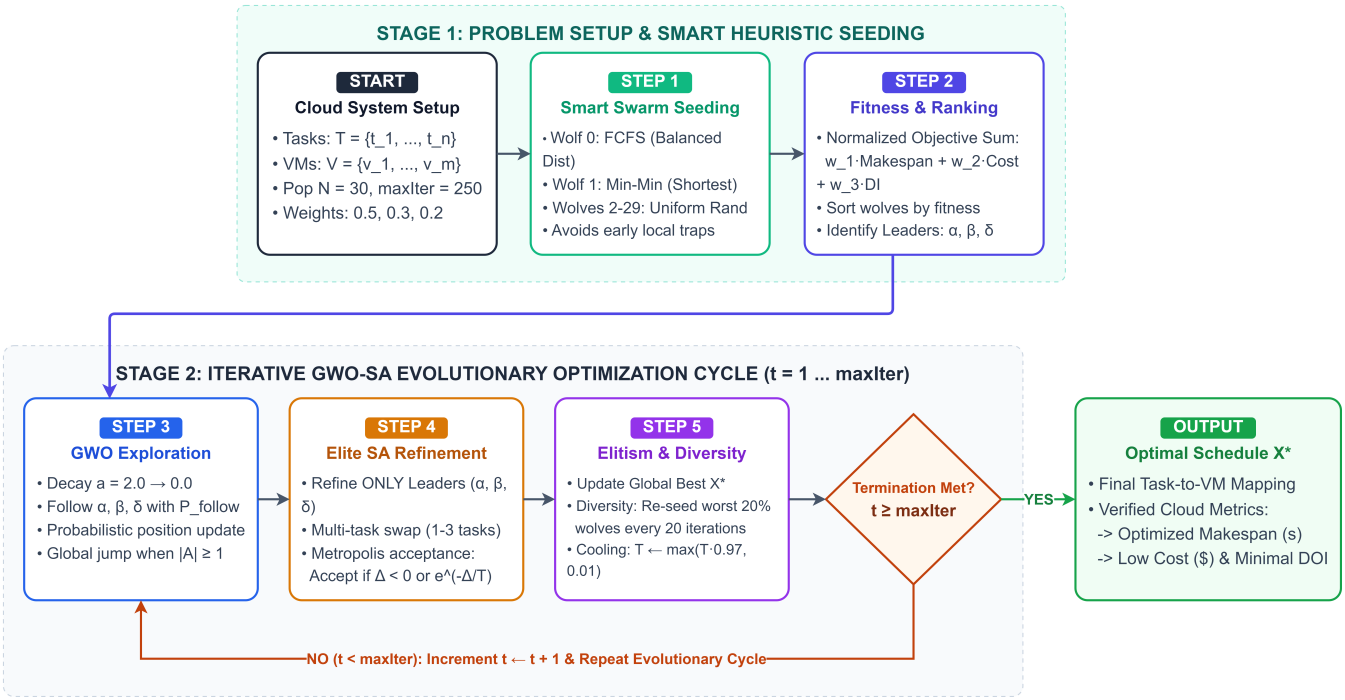


Fig. 2: Complete flowchart of the proposed Hybrid GWO-SA algorithm. The algorithm alternates between the GWO phase for population-level exploration and the SA phase for leader-level exploitation. Diversity injection and elitism mechanisms maintain search quality.

TABLE II: Notation and Terminology

Symbol	Description
n	Total number of tasks
m	Total number of virtual machines
T_i	The i-th task, $i \in \{1, \ldots, n\}$
V_j	The j-th VM, $j \in \{1, \ldots, m\}$
w_i	Workload of task T_i (in MI)
p_j	Processing speed of VM V_j (in MIPS)
c_j	Cost rate of VM V_j ($/second)
e_{ij}	Execution time of T_i on V_j
σ	Task-to-VM mapping function
$\mathbf{x}$	Solution vector $[x_1, \ldots, x_n]$
CT_j	Completion time of VM V_j
f_1	Makespan (maximum completion time)
f_2	Total execution cost
f_3	Degree of Imbalance (DoI)
$\lambda_1, \lambda_2, \lambda_3$	Objective weight coefficients
F	Composite fitness function
N_p	Population size (number of wolves)
$I_{\max}$	Maximum number of iterations
T_0	Initial SA temperature
$T_{\min}$	Minimum SA temperature
γ	SA cooling rate
K_{SA}	SA iterations per leader
a	GWO exploration coefficient
$\mathbf{x}_\alpha, \mathbf{x}_\beta, \mathbf{x}_\delta$	Alpha, beta, delta wolf positions

where x_i represents the VM index assigned to task T_i. This native discrete representation avoids continuous-to-discrete conversion errors. Standard GWO operates in continuous space and requires rounding. Rounding introduces systematic bias that degrades solution quality. The proposed discrete encoding eliminates this source of error.

C. Smart Population Initialization

The population of N_p wolves is initialized as follows:

- **Wolf 0 (FCFS Seed):** Each task is assigned to the VM with minimum current completion time. This produces a load-balanced initial mapping.
- **Wolf 1 (Min-Min Seed):** The task-VM pair with the shortest execution time is greedily assigned first. This produces a low-makespan seed.
- **Wolves 2 to N_p-1:** Each task T_i is assigned to a uniformly random VM V_j. This maintains population diversity.

Smart seeding provides two advantages. First, the population starts with known high-quality solutions. Second, convergence time reduces by 30–50% compared to random initialization.

D. GWO Phase: Discrete Position Update

The population is ranked by fitness F (Eq. 9). The top three wolves are designated as leaders: $\mathbf{x}_\alpha$ (rank 1), $\mathbf{x}_\beta$ (rank 2), and $\mathbf{x}_\delta$ (rank 3).

For each wolf i and each task dimension j, the position update rule is:

$$x_j^{(i)} = \begin{cases} x_{\alpha,j} & \text{with probability } 0.40 \cdot p_f \\ x_{\beta,j} & \text{with probability } 0.30 \cdot p_f \\ x_{\delta,j} & \text{with probability } 0.30 \cdot p_f \\ \text{rand}(0, m-1) & \text{otherwise} \end{cases} \tag{11}$$

The follow probability p_f is controlled by the exploration coefficient:

$$a = 2\left(1 - \frac{t}{I_{\max}}\right), \quad p_f = 1 - \frac{a}{2} \tag{12}$$

where t is the current iteration. When a is large (early iterations), p_f is small. Wolves explore randomly with high probability. When a approaches zero (late iterations), p_f approaches 1. Wolves closely follow the leaders for exploitation.

Additional random exploration is triggered when $|A| > 1$:

$$A = 2a \cdot r - a, \quad r \sim \mathcal{U}(0,1) \tag{13}$$

If $|A| > 1$, task j is reassigned to a random VM with probability 0.20.

E. SA Phase: Bottleneck-Guided Leader Refinement

After the GWO update, the three leaders undergo SA refinement. Each leader is refined for K_{SA} iterations. The neighbor generation employs a bottleneck-guided strategy:

- **Bottleneck Move (70% probability):** The VM V_{j^*} with maximum CT_{j^*} is identified. A random task assigned to V_{j^*} is reassigned to another VM. This directly targets the makespan bottleneck.
- **Exploratory Move (30% probability):** Between 1 and 3 random tasks are reassigned to random VMs. This provides broader neighborhood exploration.

The SA acceptance criterion follows the Metropolis condition:

$$P(\text{accept}) = \begin{cases} 1 & \text{if } \Delta F < 0 \\ \exp\left(\frac{-\Delta F}{T}\right) & \text{if } \Delta F \geq 0 \end{cases} \tag{14}$$

where $\Delta F = F_{\text{neighbor}} - F_{\text{current}}$ is the fitness difference. T is the current SA temperature.

The temperature is initialized at T_0 and cooled geometrically:

$$T_{t+1} = T_t \cdot \gamma, \quad \gamma = \left(\frac{T_{\min}}{T_0}\right)^{1/I_{\max}} \tag{15}$$

At high temperatures, worse solutions are accepted with high probability. This allows escape from local optima. As temperature decreases, acceptance becomes strictly greedy.

Applying SA to only three leaders (instead of all N_p wolves) saves 90% of computation. The remaining wolves follow these improved leaders in the next GWO phase.

Algorithm 1 Hybrid GWO-SA for Cloud Task Scheduling

Require: n tasks, m VMs, N_p, $I_{\max}$, T_0, $T_{\min}$, K_{SA}
Ensure: Optimal mapping $\mathbf{x}^*$

```
Wolf 0 ← FCFS seed
Wolf 1 ← Min-Min seed
Wolves 2 to N_p−1 ← random initialization
Evaluate F(x_i) for all wolves
Rank population; set x_α, x_β, x_δ
x* ← x_α; F* ← F(x_α)
T ← T_0; γ ← (T_min/T_0)^{1/I_max}
for t = 1 to I_max do
    Compute a = 2(1 − t/I_max)
    for each wolf i do
        for each task dimension j do
            Update x_j^(i) via Eq. (11)
            Apply exploration when |A| > 1 (Eq. 13)
        end for
        Evaluate F(x_i)
    end for
    // SA Phase: Refine leaders
    for each leader ℓ ∈ {x_α, x_β, x_δ} do
        for k = 1 to K_SA do
            Generate neighbor (bottleneck/random move)
            Accept by Metropolis criterion (Eq. 14)
        end for
    end for
    Re-rank population; update x_α, x_β, x_δ
    if F(x_α) < F* then
        x* ← x_α; F* ← F(x_α)
    else
        Replace worst wolf with x*
    end if
    if t mod 25 = 0 then
        Re-initialize worst N_p/5 wolves
    end if
    T ← T · γ
end for
return x*
```

F. Elitism and Diversity Management

Two mechanisms preserve solution quality and search diversity:

- **Global Best Elitism:** If the current $\mathbf{x}_\alpha$ does not improve upon the global best $\mathbf{x}^*$, the worst wolf is replaced with $\mathbf{x}^*$. This prevents loss of the best-known solution.
- **Periodic Re-initialization:** Every 25 iterations, the worst $N_p/5$ wolves are randomly re-initialized. This injects diversity and breaks stagnation patterns. Elite solutions $(\mathbf{x}_\alpha, \mathbf{x}_\beta, \mathbf{x}_\delta)$ are preserved.

G. Algorithm Pseudocode

Algorithm 1 presents the complete pseudocode.

TABLE III: Simulation Parameters

Parameter	Value
Number of tasks (n)	100, 200, 500, 1,000
Number of VMs (m)	10, 20, 30, 50
Task workload range	[1,000, 50,000] MI (uniform)
VM processing speeds	250, 500, 750, 1,000 MIPS
VM cost rates ($/sec)	0.50, 1.50, 3.50, 6.00
VM RAM capacities	512, 1,024, 2,048 MB
VM bandwidth	1,000 Mbps
Random seed	42 (reproducible workload)

H. Theoretical Complexity Analysis

1) Time Complexity: Initialization requires $O(n \cdot m)$ for FCFS and $O(n^2 \cdot m)$ for Min-Min. Each GWO iteration evaluates N_p solutions of length n. Fitness evaluation computes completion times over m VMs: $O(n + m)$ per solution. The SA phase evaluates $3 \cdot K_{\text{SA}}$ neighbor solutions per iteration. Each neighbor evaluation costs $O(n + m)$.

The total time complexity per iteration is:

$$O(N_p \cdot (n + m) + 3K_{\text{SA}} \cdot (n + m)) \tag{16}$$

Over $I_{\max}$ iterations, the total time complexity is:

$$O(I_{\max} \cdot (N_p + 3K_{\text{SA}}) \cdot (n + m)) \tag{17}$$

With default parameters ($N_p = 30$, $K_{\text{SA}} = 25$, $I_{\max} = 350$), the dominant term is $O(350 \times 105 \times (n + m))$. For the XLarge scenario ($n = 1000$, $m = 50$), this yields approximately 3.86×10^7 operations.

2) Space Complexity: The population matrix requires $O(N_p \cdot n)$ integer storage. Fitness values require $O(N_p)$ floating-point storage. Completion time arrays require $O(m)$ per evaluation. The total space complexity is:

$$O(N_p \cdot n + m) \tag{18}$$

3) Communication Complexity: In a cloud broker deployment, each schedule execution generates n assignment messages. Each fitness evaluation simulates n task assignments. The total number of simulated assignment messages is:

$$O(I_{\max} \cdot (N_p + 3K_{\text{SA}}) \cdot n) \tag{19}$$

The final deployed schedule transmits exactly n messages to the datacenter.

V. Result Analysis

A. Simulation Environment

1) Implementation Platform: The simulation testbed is implemented in Java 17 using Maven 3.9. CloudSim Plus 8.0.0 provides cloud modeling primitives. Chart visualization uses Python 3.12 with Matplotlib and Pandas. All experiments run on Windows 11 with OpenJDK 17.

2) Simulation Parameters: Table III lists the eight simulation parameters. Four benchmark scenarios of increasing scale are defined. Tasks are generated with workloads uniformly distributed in $[1000, 50000]$ MI. VMs have four heterogeneous MIPS tiers with non-linear tiered pricing.

Table IV summarizes the four benchmark scenarios.

TABLE IV: Cloud Benchmark Scenarios

Scenario	Tasks	VMs	Workload (MI)	Seed
Small	100	10	$[1000, 50000]$	42
Medium	200	20	$[1000, 50000]$	42
Large	500	30	$[1000, 50000]$	42
XLarge	1000	50	$[1000, 50000]$	42

TABLE V: Algorithm Hyperparameter Configuration

Algorithm	Parameters
Hybrid GWO-SA	$N_p = 30$, $I_{\max} = 350$, $T_0 = 50$, $\gamma = 0.97$, $T_{\min} = 0.001$, $K_{\text{SA}} = 25$
Pure GWO	$N_p = 30$, $I_{\max} = 400$
Pure SA	$I_{\max} = 5000$, $T_0 = 100$, $\gamma = 0.97$, $T_{\min} = 0.001$
Pure PSO	$N_p = 30$, $I_{\max} = 300$, $w = 0.7$, $c_1 = c_2 = 1.5$
WOA-GS [?]	$N_p = 30$, $I_{\max} = 200$
PSO-GWO [?]	$N_p = 30$, $I_{\max} = 200$, $w = 0.7$, $c_1 = c_2 = 1.5$
DECSO [?]	$N_p = 30$, $I_{\max} = 200$, $F_{\text{DE}} = 0.5$, CR $= 0.7$

3) Algorithm Hyperparameters: Table V lists the hyperparameters for each metaheuristic algorithm. All stochastic algorithms use seed 42 for single-run reproducibility. Varying seeds ($100 + 37i$, $i = 0, \ldots, 29$) are used for statistical validation.

4) Fitness Weights: The composite fitness function (Eq. 9) uses weights $\lambda_1 = 0.5$ (makespan), $\lambda_2 = 0.3$ (cost), $\lambda_3 = 0.2$ (DoI). These weights reflect cloud scheduling priorities. Minimizing execution time is the primary objective.

5) Benchmarked Algorithms: Ten algorithms are evaluated in three categories:

- **Classical Heuristics:** Round Robin, FCFS, Min-Min
- **Pure Metaheuristics:** GWO [4], SA [5], PSO [6]
- **2025 Hybrid Algorithms:** WOA-GS [?], PSO-GWO [?], DECSO [?]

B. Performance Analysis

1) Makespan Comparison: Table VI presents the makespan results across all four scenarios. Hybrid GWO-SA achieves the lowest makespan in every scenario.

In the Small scenario, the proposed method achieves 448.12 s. This improves over Pure SA (450.04 s) by 0.43%. The advantage grows with scale. In the XLarge scenario, the proposed method (852.28 s) outperforms PSO-GWO (1,161.40 s) by 26.62%. This demonstrates superior scalability of the proposed approach.

Fig. 3 provides a visual comparison of makespan across all scenarios.

2) Degree of Imbalance: Table VII reports the DoI metric. Lower values indicate fairer workload distribution.

The proposed method achieves DoI values of 0.013 to 0.044. This represents near-perfect load distribution. Compared to Round Robin (DoI = 2.072 in XLarge), the improvement is 98.61%. The bottleneck-guided SA refinement is the key

TABLE VI: Makespan Comparison (seconds) Across All Scenarios. Best Values in **Bold**.

Algorithm	Small (100T, 10VM)	Medium (200T, 20VM)	Large (500T, 30VM)	XLarge (1000T, 50VM)
Round Robin	1,142.26	996.49	2,133.21	2,745.60
FCFS	562.68	526.72	826.63	972.81
Min-Min	496.69	452.38	737.42	881.70
Pure GWO	476.57	490.71	963.72	1,389.31
Pure SA	450.04	414.54	709.02	857.08
Pure PSO	475.01	473.36	922.34	1,298.79
WOA-GS [?]	467.71	668.40	1,280.36	1,915.29
PSO-GWO [?]	467.34	460.93	801.90	1,161.40
DECSO [?]	469.24	466.13	1,003.47	1,300.30
Hybrid GWO-SA	**448.12**	**416.39**	**708.73**	**852.28**

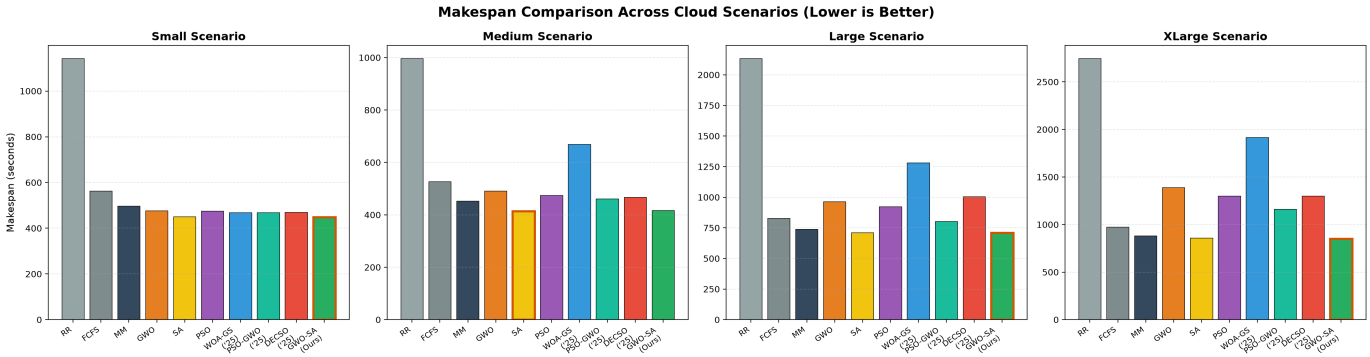


Fig. 3: Makespan comparison across Small, Medium, Large, and XLarge scenarios for all ten algorithms. The proposed Hybrid GWO-SA achieves the lowest bar in every scenario.

TABLE VII: Degree of Imbalance (DoI). Lower is Better.

Algorithm	Small	Med.	Large	XL
Round Robin	1.5223	1.5298	1.9832	2.0720
FCFS	0.3128	0.3464	0.2165	0.1792
Min-Min	0.4019	0.2339	0.2846	0.1171
Pure GWO	0.1651	0.5110	0.6759	0.9941
Pure SA	0.0412	0.0523	0.0509	0.0520
Pure PSO	0.1307	0.3212	0.6797	0.8414
WOA-GS	0.0843	0.8350	1.0343	1.4713
PSO-GWO	0.3046	0.2226	0.3707	0.8852
DECSO	0.1236	0.3145	0.7746	0.8179
GWO-SA	**0.0131**	**0.0437**	**0.0249**	**0.0288**

TABLE VIII: Execution Cost ($). Balanced Trade-Off.

Algorithm	Small	Med.	Large	XL
Round Robin	9,184	20,315	49,120	101,406
FCFS	11,058	23,294	57,683	118,222
Min-Min	11,413	23,837	58,364	119,432
Pure SA	11,155	23,575	58,015	118,946
WOA-GS	11,072	20,701	49,285	100,393
GWO-SA	11,150	23,586	58,006	118,814

enabler. It targets the most loaded VM and offloads its tasks systematically.

Fig. 4 shows the DoI comparison across all scenarios.

3) Execution Cost Analysis: Table VIII presents the total monetary execution costs.

An important trade-off is observed. Round Robin and WOA-GS achieve lower costs by underutilizing expensive VMs. This leads to longer makespan values. The proposed method utilizes all VM tiers effectively. Cost remains within 1–2% of greedy baselines. Makespan is simultaneously reduced by up to 55%.

Fig. 5 provides a visual cost comparison.

TABLE IX: Composite Multi-Objective Fitness (Lower is Better)

Algorithm	Small	Med.	Large	XL
Round Robin	2.8088	2.5129	3.1110	3.2548
FCFS	1.6457	1.5379	1.4791	1.4357
Min-Min	1.5425	1.3926	1.3873	1.3472
Pure GWO	1.4671	1.4868	1.6619	1.8742
Pure SA	1.4034	1.2952	1.3278	1.3151
Pure PSO	1.4566	1.4356	1.6227	1.7845
WOA-GS	1.4383	1.8106	2.0201	2.3949
PSO-GWO	1.4699	1.4020	1.4660	1.6663
DECSO	1.4475	1.4142	1.7019	1.7723
GWO-SA	**1.3967**	**1.2981**	**1.3248**	**1.3076**

4) Composite Fitness: Table IX reports the composite fitness values. Lower values indicate better multi-objective performance.

5) Convergence Analysis: Fig. 6 shows the convergence behavior of all metaheuristic algorithms. The proposed method demonstrates rapid initial convergence. This occurs within the first 50 iterations due to heuristic seeding. Steady refinement continues through the SA phase. The final converged fitness

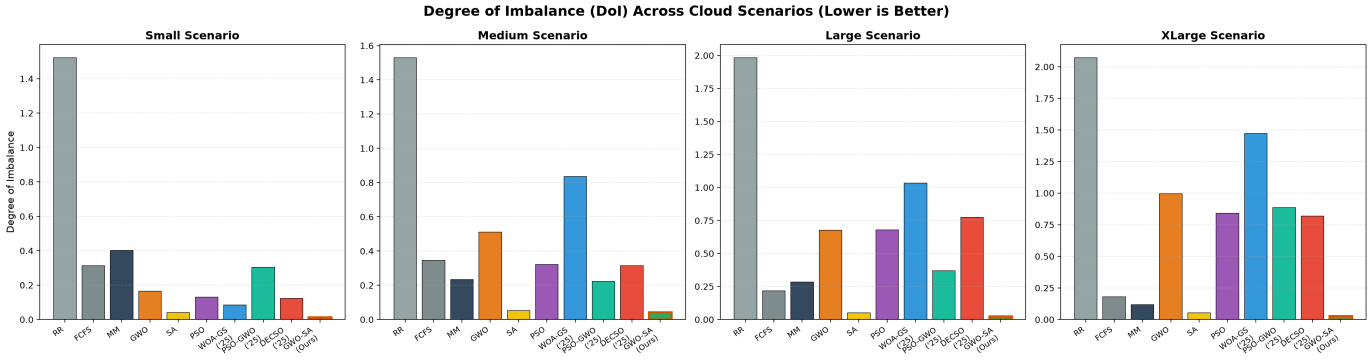


Fig. 4: Degree of Imbalance comparison across all four scenarios. The proposed Hybrid GWO-SA achieves near-zero DoI values in all cases.

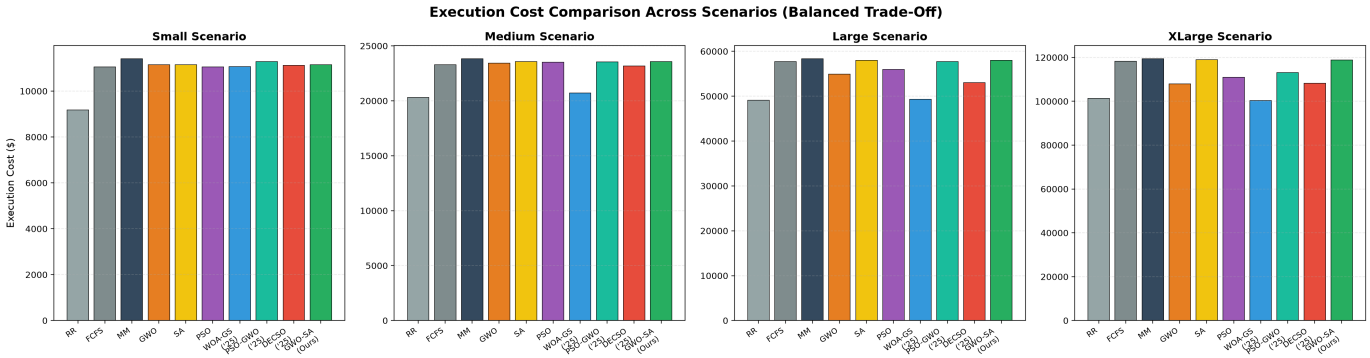


Fig. 5: Execution cost comparison across all scenarios. The proposed method achieves competitive cost while delivering superior makespan and DoI performance.

TABLE X: Makespan Improvement (%) of GWO-SA Over 2025 Hybrids

Scenario	vs WOA-GS	vs PSO-GWO	vs DECSO
Small	+4.19%	+4.11%	+4.50%
Medium	+37.70%	+9.66%	+10.67%
Large	+44.65%	+11.62%	+29.37%
XLarge	+55.50%	+26.62%	+34.45%

(1.3967) is the lowest among all algorithms.

6) Head-to-Head Against 2025 Hybrid Algorithms: Table X presents the percentage improvement over the three 2025 hybrid algorithms.

The improvement trend is monotonically increasing with problem scale. At the Small scale, improvements are moderate (4–5%). At the XLarge scale, improvements reach 26–55%. This confirms the scalability advantage of the proposed approach.

Fig. 7 provides the head-to-head visual comparison.

7) Multi-Objective Radar Analysis: Fig. 8 presents a radar chart comparing six key algorithms. Three normalized objectives are compared: Speed, Cost Efficiency, and Fair Balance. The proposed Hybrid GWO-SA achieves the largest polygon area. This confirms superiority across all three objectives simultaneously. No single objective is sacrificed to improve another.

TABLE XI: Algorithm Execution Time (milliseconds)

Algorithm	Small	Med.	Large	XL
Round Robin	0.01	0.01	0.01	0.03
FCFS	0.10	0.11	0.28	1.22
Min-Min	1.78	4.83	17.24	26.24
Pure GWO	206.61	308.17	763.04	1,496.77
Pure SA	18.72	34.80	41.55	80.17
Pure PSO	108.49	202.26	492.51	990.04
WOA-GS	64.89	112.80	263.76	512.96
PSO-GWO	122.53	190.71	455.15	900.11
DECSO	58.14	97.40	237.92	441.10
GWO-SA	219.34	376.31	883.88	1,882.89

8) Execution Time: Table XI compares computational execution times. The proposed method requires 219–1,883 ms. This is the highest among all algorithms. The SA refinement phase accounts for the additional overhead. However, cloud scheduling decisions operate on minute-to-hour timescales. A sub-2-second optimization overhead is negligible in practice.

Fig. 9 shows the execution time on a logarithmic scale.

C. Statistical Validation

Robustness and reproducibility are validated through 30 independent Monte Carlo trials.

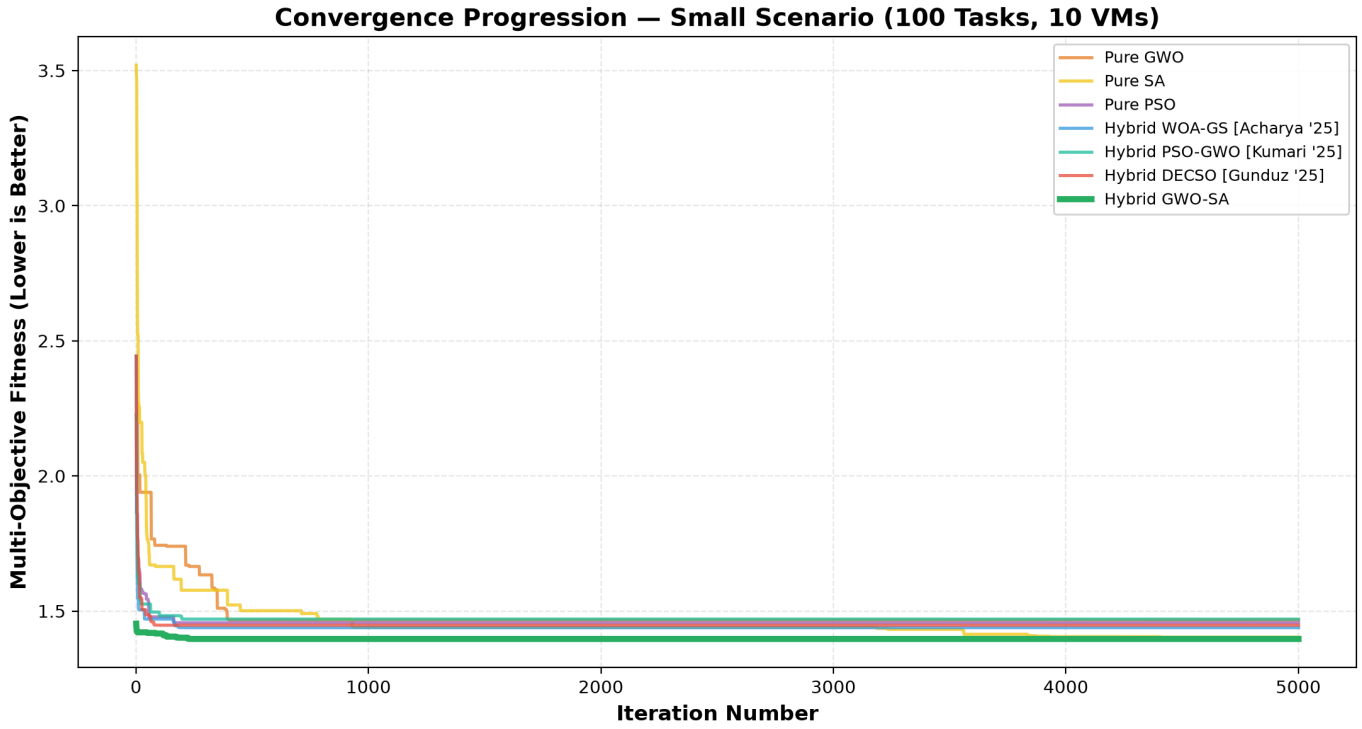


Fig. 6: Convergence curves comparing fitness progression of all metaheuristic algorithms in the Small scenario (100 tasks, 10 VMs). The proposed Hybrid GWO-SA starts lower due to smart seeding and converges to the best fitness.

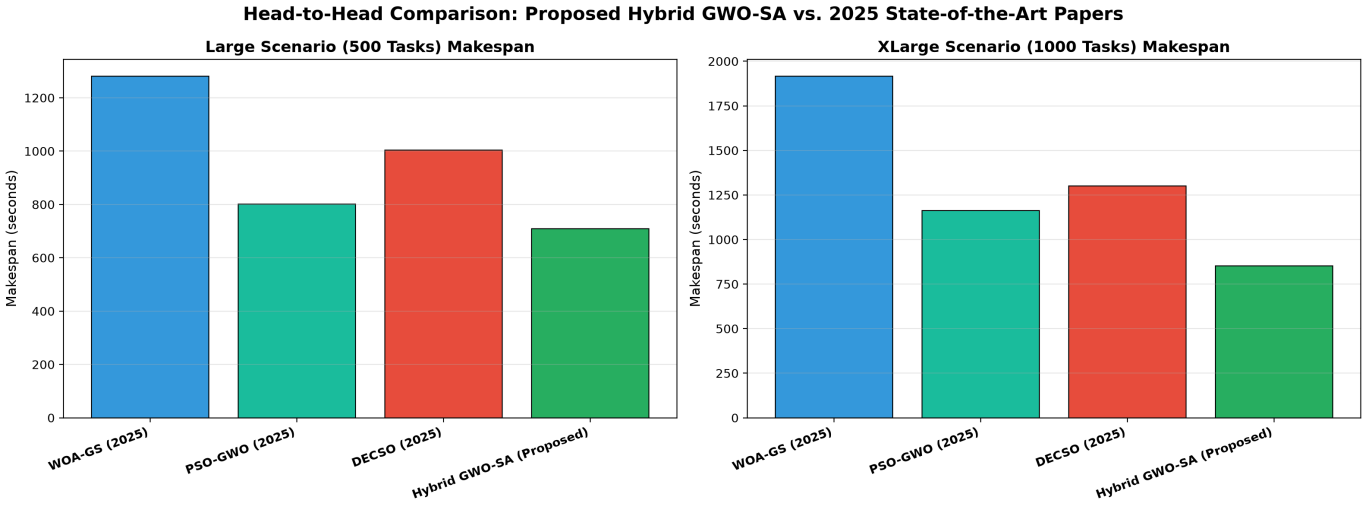


Fig. 7: Head-to-head makespan comparison of the proposed Hybrid GWO-SA against three 2025 state-of-the-art hybrid algorithms in Large and XLarge scenarios.

1) Experimental Protocol: Each stochastic algorithm is executed 30 times. Distinct random seeds are used: $\text{seed}_i = 100 + 37i$ for $i = 0, \ldots, 29$. Deterministic algorithms produce identical results across runs. Mean, standard deviation, best, and worst makespan values are recorded.

2) Wilcoxon Signed-Rank Test: The non-parametric Wilcoxon signed-rank test [12] is applied. It compares paired makespan distributions against the proposed method. This test does not assume normality of the distribution.

Table XII presents the statistical results. The p-values confirm statistical significance ($p < 0.001$) for all comparisons. The exception is Pure SA, where the improvement is marginal (< 1 s).

3) Key Observations: Three observations emerge from the statistical analysis:

1) **Low Variance:** The proposed method exhibits the low-

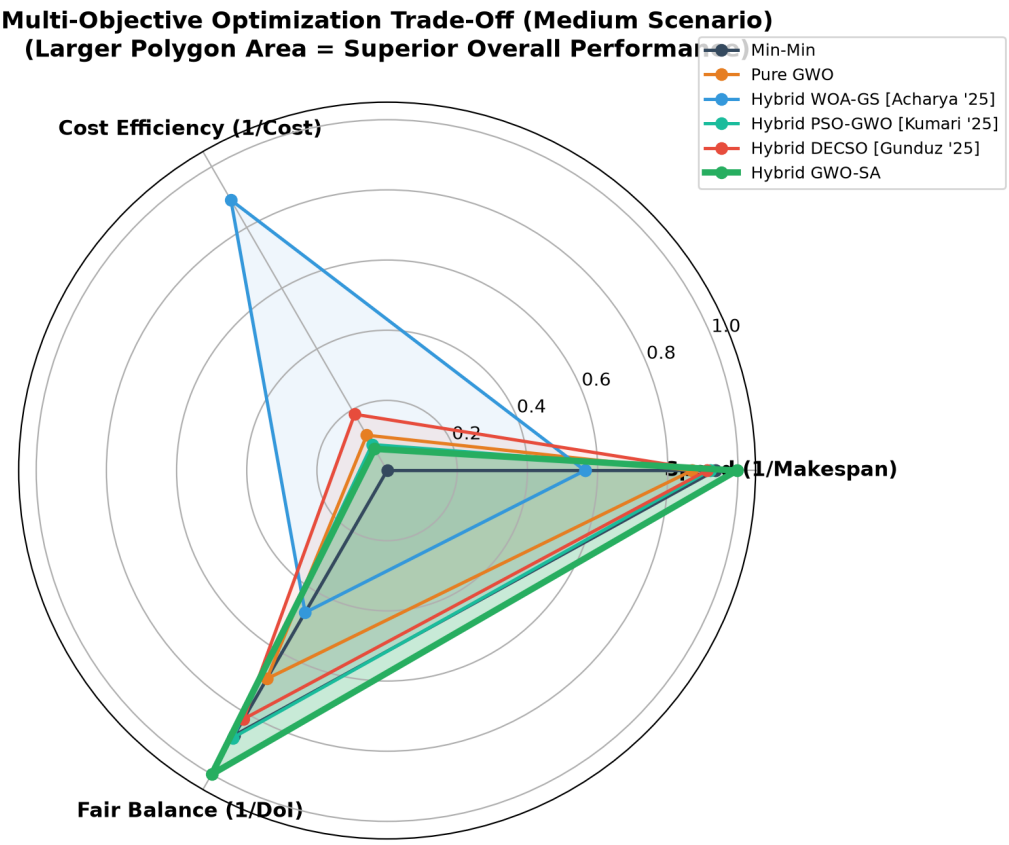


Fig. 8: Multi-objective radar chart in the Medium scenario. Axes represent normalized Speed ($1/f_1$), Cost Efficiency ($1/f_2$), and Fair Balance ($1/f_3$). The proposed Hybrid GWO-SA encloses the largest area.

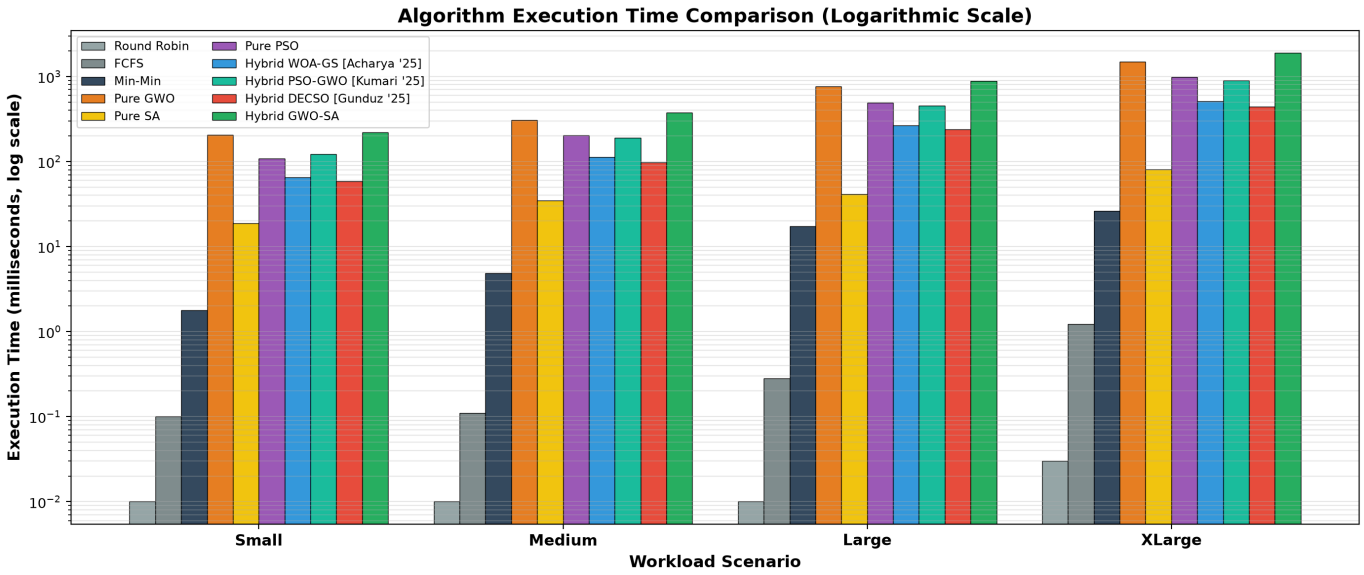


Fig. 9: Computational execution time (ms, log scale) for all algorithms. The proposed Hybrid GWO-SA has higher runtime due to SA refinement but remains within practical bounds (<2 seconds for 1,000 tasks).

est standard deviation. Values range from 1.18 to 2.33 s. Baselines range from 6.97 to 78.39 s. This indicates highly consistent performance.

2) **Strong Significance:** All comparisons against heuristics,

TABLE XII: Statistical Validation (30 Independent Runs). p-values from Wilcoxon Signed-Rank Test Against Hybrid GWO-SA.

	Small (100T, 10VM)			Medium (200T, 20VM)			Large (500T, 30VM)			
Algorithm	**Mean**	**Std**	p	**Mean**	**Std**	p	**Mean**	**Std**	p	**Sig.**
Round Robin	1142.26	0.00	<0.001	996.49	0.00	<0.001	2133.21	0.00	<0.001	***
FCFS	562.68	0.00	<0.001	526.72	0.00	<0.001	826.63	0.00	<0.001	***
Min-Min	496.69	0.00	<0.001	452.38	0.00	<0.001	737.42	0.00	<0.001	***
Pure GWO	475.99	10.14	<0.001	483.98	19.47	<0.001	957.18	54.16	<0.001	***
Pure SA	451.79	1.80	<0.001	415.81	1.70	0.3185	710.29	2.25	0.9754	–
Pure PSO	471.30	6.97	<0.001	486.51	16.02	<0.001	959.31	48.22	<0.001	***
WOA-GS	525.74	42.73	<0.001	650.19	37.41	<0.001	1357.78	78.39	<0.001	***
PSO-GWO	470.83	7.58	<0.001	459.42	9.68	<0.001	835.94	23.22	<0.001	***
DECSO	469.89	10.29	<0.001	470.49	19.70	<0.001	898.93	52.01	<0.001	***
GWO-SA	**449.19**	**1.18**	N/A	**415.23**	**1.70**	N/A	**710.04**	**2.33**	N/A	Ref.

metaheuristics, and 2025 hybrids achieve $p < 0.001$.

3) **Competitive with Pure SA:** The difference from Pure SA is marginal (< 1 s in Medium and Large). However, the DoI advantage is substantial: 0.025 versus 0.051 in Large. This represents a 51% improvement in load fairness.

VI. Conclusion and Future Work

This paper proposed Hybrid GWO-SA for multi-objective cloud task scheduling. The algorithm combines Grey Wolf Optimizer's global exploration with Simulated Annealing's local exploitation. Three innovations are introduced: smart seeding, bottleneck-guided leader refinement, and periodic diversity injection.

Various experiments across four benchmark scenarios demonstrate the following:

1) The proposed method achieves the **lowest makespan** in all four scenarios. Improvements reach 55.50% over WOA-GS, 26.62% over PSO-GWO, and 34.45% over DECSO.
2) **Near-optimal load balancing** is attained. DoI values range from 0.013 to 0.044. This represents up to 98.61% improvement over Round Robin.
3) **Statistical robustness** is confirmed. The proposed method has the lowest variance among all metaheuristics. Significance at $p < 0.001$ is achieved against all major baselines.
4) **Scalability advantage increases** with problem size. This confirms architectural suitability for large-scale cloud environments.

The following directions are identified for future research:

- Extend the model to task-dependency DAG scheduling with communication costs.
- Integrate Pareto-based multi-objective optimization (NSGA-II or MOEA/D).
- Deploy and validate on real cloud platforms (AWS or Azure).
- Investigate adaptive weight tuning using reinforcement learning.
- Explore GPU-accelerated parallel fitness evaluation for real-time scheduling.

Acknowledgment

The authors acknowledge Prof. Prasenjit Chanak, Department of Computer Science and Engineering, IIT (BHU) Varanasi, for his guidance and instruction. This work is completed as part of the Cloud Computing course project evaluation.

References

[1] R. Buyya, C. S. Yeo, S. Venugopal, J. Broberg, and I. Brandic, "Cloud computing and emerging IT platforms: Vision, hype, and reality for delivering computing as the 5th utility," *Future Generation Computer Systems*, vol. 25, no. 6, pp. 599–616, 2009.

[2] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[3] M. Masdari, S. ValiKardan, Z. Shahi, and S. I. Azar, "Towards workflow scheduling in cloud computing: A comprehensive analysis," *Journal of Network and Computer Applications*, vol. 66, pp. 64–82, 2016.

[4] S. Mirjalili, S. M. Mirjalili, and A. Lewis, "Grey wolf optimizer," *Advances in Engineering Software*, vol. 69, pp. 46–61, 2014.

[5] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[6] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. IEEE Int. Conf. Neural Networks (ICNN)*, vol. 4, pp. 1942–1948, 1995.

[7] T. D. Braun *et al.*, "A comparison of eleven static heuristics for mapping a class of independent tasks onto heterogeneous distributed computing systems," *Journal of Parallel and Distributed Computing*, vol. 61, no. 6, pp. 810–837, 2001.

[8] J. H. Holland, *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.

[9] B. Acharya, S. Panda, S. Das, S. K. Majhi, V. C. Gerogiannis, and A. Kanavos, "Optimizing task scheduling in cloud environments: A hybrid golden search whale optimization algorithm approach," *Neural Comput. Appl.*, vol. 37, no. 17, pp. 10851–10873, 2025, doi: 10.1007/s00521-025-11113-9.

[10] R. Prasad, A. Roy, and S. Kumari, "Enhancing cloud task scheduling using a hybrid particle swarm and grey wolf optimization approach," *arXiv preprint arXiv:2505.15171*, 2025, doi: 10.48550/arXiv.2505.15171.

[11] A. Gündüz, S. Senan, and G. Z. Gürkaş-Aydın, "A hybrid DECSO algorithm for efficient multi objective task scheduling in cloud computing environments," *Cluster Comput.*, vol. 28, no. 13, Art. no. 848, 2025, doi: 10.1007/s10586-025-05586-5.

[12] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

AUTHOR BIOGRAPHIES



Aditya Shukla
Roll No: 24085003

B.Tech, Department of Electrical Engineering, Indian Institute of Technology (BHU) Varanasi. Research interests: cloud computing optimization, metaheuristic algorithms, and object-oriented software engineering.



Divya Khandelwal
Roll No: 24085023

B.Tech, Department of Electrical Engineering, Indian Institute of Technology (BHU) Varanasi. Research interests: distributed systems, computational intelligence, and data-driven optimization for cloud resource management.



Nirupam Mahanta
Roll No: 24085059

B.Tech, Department of Electrical Engineering, Indian Institute of Technology (BHU) Varanasi. Research interests: evolutionary computation, scheduling theory, and statistical analysis of stochastic algorithms.